

// Jivops

Observabilidad de IA

Guía completa — 30 lecciones

Fundamentos · Tracing · Métricas y calidad · Costos y rendimiento · Alertas e incidentes · Producción avanzada

Contenido

Módulo 1: Fundamentos de observabilidad para sistemas de IA

- Lección 01 — Por qué la observabilidad tradicional no es suficiente para sistemas de IA
- Lección 02 — Los tres pilares aplicados a IA — logs, métricas y trazas
- Lección 03 — Qué hace única a la observabilidad de IA — calidad y semántica
- Lección 04 — OpenTelemetry y estandarización de trazas para IA
- Lección 05 — Arquitectura de un stack de observabilidad de IA

Módulo 2: Tracing de sistemas de IA

- Lección 06 — Trazas de extremo a extremo en una aplicación LLM/agente
- Lección 07 — Spans anidados — de la petición del usuario a cada llamada al modelo
- Lección 08 — Herramientas de tracing especializadas (LangSmith, Langfuse, Arize)
- Lección 09 — Correlación de trazas entre microservicios y llamadas al LLM
- Lección 10 — Sampling de trazas — qué capturar a gran escala

Módulo 3: Métricas y monitoreo de calidad

- Lección 11 — Métricas de negocio vs métricas técnicas en sistemas de IA
- Lección 12 — Monitoreo de calidad de respuesta en producción
- Lección 13 — Detección de drift de datos y de modelo en tiempo real
- Lección 14 — Métricas de seguridad — tasa de bloqueo de guardrails, intentos de jailbreak
- Lección 15 — Dashboards efectivos para sistemas de IA

Módulo 4: Costos y rendimiento

- Lección 16 — Observabilidad de costos — tokens, llamadas, por usuario/feature
- Lección 17 — Atribución de costos en sistemas multi-modelo
- Lección 18 — Latencia — percentiles y su impacto en experiencia de usuario
- Lección 19 — Observabilidad de uso de GPU y recursos de inferencia
- Lección 20 — Presupuestos y alertas de costos

Módulo 5: Alertas, anomalías y respuesta a incidentes

- Lección 21 — Definir alertas significativas para sistemas de IA
- Lección 22 — Detección de anomalías en el comportamiento del modelo
- Lección 23 — Runbooks para incidentes específicos de IA
- Lección 24 — On-call para sistemas de IA — qué es diferente
- Lección 25 — Post-mortems de incidentes de IA

Módulo 6: Producción avanzada

Lección 26 — Feedback humano como señal de observabilidad

Lección 27 — Observabilidad para cumplimiento y auditoría (compliance)

Lección 28 — Observabilidad multi-agente — visibilidad de sistemas complejos

Lección 29 — Cultura de observabilidad — de reactivo a proactivo

Lección 30 — Proyecto final: stack de observabilidad de IA completo listo para producción

Módulo 1: Fundamentos de observabilidad para sistemas de IA

Lección 01 — Por qué la observabilidad tradicional no es suficiente para sistemas de IA

No captura si la respuesta generada es de buena calidad, factualmente correcta o útil, ya que un sistema puede estar 'sano' técnicamente (responde rápido, sin errores HTTP) mientras genera respuestas de mala calidad.

La latencia y los errores HTTP miden si el sistema respondió rápido y sin fallos técnicos, pero no dicen nada sobre si el contenido de esa respuesta era correcto o útil; un modelo podría generar una alucinación completa en 200 milisegundos sin ningún error HTTP, y ese dashboard tradicional lo mostraría como un sistema perfectamente saludable.

Lección 02 — Los tres pilares aplicados a IA — logs, métricas y trazas

Los logs capturan el prompt y la respuesta generada, las métricas incluyen indicadores de calidad además de los técnicos, y las trazas siguen el flujo completo desde la petición del usuario hasta cada llamada individual al modelo.

El sistema podría transformar significativamente la pregunta original del usuario (añadiendo contexto recuperado, instrucciones del sistema) antes de enviarla realmente al modelo; sin registrar ese prompt final exacto, sería imposible reconstruir después qué información realmente vio el modelo al generar esa respuesta específica que resultó problemática.

Lección 03 — Qué hace única a la observabilidad de IA — calidad y semántica

La evaluación de la calidad semántica del contenido generado (si es correcto, relevante, coherente), una dimensión subjetiva y más difícil de medir automáticamente que un simple código de estado HTTP.

Un código de error HTTP 500 es una señal binaria objetiva y fácil de verificar automáticamente (el servidor falló o no falló); determinar si una respuesta de texto generada es 'correcta' requiere evaluar significado, contexto y a veces no hay una única respuesta correcta verificable de forma tan simple y directa como comparar un código numérico.

Lección 04 — OpenTelemetry y estandarización de trazas para IA

Permite que las trazas generadas sean compatibles con múltiples herramientas de observabilidad distintas, evitando quedar atado a un único proveedor específico y facilitando cambiar de herramienta de análisis en el futuro sin reinstrumentar todo el código.

Si las trazas ya siguen el formato estándar de OpenTelemetry, cambiar de herramienta de análisis (de una plataforma a otra) solo requiere redirigir hacia dónde se envían esos datos ya estandarizados, sin tener que reescribir todo el código de instrumentación original, algo que sí sería necesario si se hubiera usado un formato propietario específico de la herramienta anterior.

Lección 05 — Arquitectura de un stack de observabilidad de IA

Recolección de trazas y logs específicos de IA, almacenamiento y agregación de esos datos, un sistema de evaluación de calidad (posiblemente con LLM-as-judge), y visualización mediante dashboards junto con alertas configuradas.

Las trazas y logs capturan fielmente qué ocurrió técnicamente (qué prompt se envió, qué respuesta se generó), pero por sí solos no dicen si esa respuesta generada fue realmente buena o mala; un componente dedicado de evaluación de calidad es lo que añade ese juicio adicional necesario, conectando los datos técnicos capturados con una valoración real de la calidad obtenida.

Módulo 2: Tracing de sistemas de IA

Lección 06 — Trazas de extremo a extremo en una aplicación LLM/agente

Todo el recorrido de una petición individual, desde que el usuario envía su pregunta, pasando por la búsqueda de documentos relevantes, la construcción del prompt, hasta la respuesta final generada y devuelta al usuario.

Una traza conectada muestra el tiempo real que tomó cada etapa específica (recuperación, construcción de prompt, generación) dentro del contexto de esa misma petición completa, permitiendo atribuir claramente dónde se concentró la lentitud; logs aislados de cada componente por separado, sin esa conexión explícita, dificultarían mucho más determinar cuál etapa específica fue realmente responsable del retraso total observado.

Lección 07 — Spans anidados — de la petición del usuario a cada llamada al modelo

Cada span representa una unidad de trabajo específica (como una llamada al LLM o una búsqueda vectorial) que puede contener otros spans hijos dentro de sí, reflejando la estructura real de sub-pasos dentro de una operación mayor.

El span padre de 'recuperación de contexto' muestra el tiempo total de toda esa etapa, mientras el span hijo anidado de 'búsqueda vectorial' específicamente muestra cuánto de ese tiempo total correspondió solo a esa sub-operación particular, dando visibilidad tanto del resumen general como del detalle específico según se necesite profundizar.

Lección 08 — Herramientas de tracing especializadas (LangSmith, Langfuse, Arize)

Ofrece visualizaciones y funcionalidades diseñadas específicamente para el contexto de IA, como mostrar el prompt y la respuesta completa de cada llamada al modelo, comparar versiones de prompts, o calcular el costo en tokens directamente en la interfaz.

En una herramienta especializada, el prompt exacto y la respuesta completa de cada llamada aparecen ya organizados y listos para revisar directamente en la interfaz; con logs genéricos dispersos, habría que buscar, extraer y correlacionar manualmente esa misma información desde distintas fuentes, un proceso mucho más lento y propenso a error al momento de diagnosticar un problema de calidad específico.

Lección 09 — Correlación de trazas entre microservicios y llamadas al LLM

Requiere propagar un identificador único de traza (trace ID) consistentemente a través de todos los microservicios involucrados, de forma que cada uno pueda reportar su parte de la traza bajo ese mismo identificador compartido.

Sin un identificador compartido y consistente, cada microservicio registraría su actividad de forma completamente aislada, sin ninguna forma explícita de vincular esos eventos dispersos entre sí como parte de la misma petición original del usuario; el trace ID propagado es precisamente lo que permite reconectar esos fragmentos dispersos en una traza única y coherente de extremo a extremo.

Lección 10 — Sampling de trazas — qué capturar a gran escala

Capturar y almacenar la traza completa de cada petición individual a gran escala puede generar un volumen de datos y un costo de almacenamiento significativo, así que se captura una muestra

representativa que sigue dando visibilidad útil sin ese costo completo.

Los casos con errores son típicamente los que el equipo más necesita investigar en detalle para diagnosticar problemas; un muestreo completamente aleatorio sin ese criterio de priorización corre el riesgo de, por pura casualidad estadística, no capturar precisamente esos casos problemáticos más valiosos de investigar, mientras una estrategia que prioriza los errores garantiza tener siempre esa información disponible cuando realmente se necesita.

Módulo 3: Métricas y monitoreo de calidad

Lección 11 — Métricas de negocio vs métricas técnicas en sistemas de IA

La métrica técnica mide el rendimiento operativo del sistema en sí, mientras la métrica de negocio mide si el sistema realmente está logrando el objetivo real por el cual se construyó, más allá de si funciona técnicamente bien.

Aunque el sistema responde técnicamente muy rápido, si el 80% de las veces esa respuesta rápida no logra realmente resolver la consulta del usuario (terminando escalada a un humano de todas formas), el propósito real de negocio del sistema —reducir la carga sobre el equipo humano— estaría fallando, sin importar lo excelente que sea su métrica técnica de velocidad.

Lección 12 — Monitoreo de calidad de respuesta en producción

Aplicar evaluación automatizada (como LLM-as-judge) sobre una muestra continua del tráfico real de producción, calculando métricas de calidad de forma sistemática y a escala, complementada ocasionalmente con revisión humana de casos específicos.

Si la muestra es suficientemente grande y representativa del tráfico total, las tendencias de calidad que revela (mejorando, empeorando, estable) reflejan razonablemente bien el comportamiento general del sistema completo, sin necesitar el costo adicional de evaluar absolutamente cada respuesta individual generada, que aportaría relativamente poco valor adicional sobre esa tendencia ya visible en la muestra.

Lección 13 — Detección de drift de datos y de modelo en tiempo real

Detectar un cambio significativo en la distribución de los datos de entrada tan pronto como comienza a ocurrir, permitiendo una respuesta más rápida antes de que la degradación de calidad se acumule y afecte a muchos más usuarios.

Si el drift comienza a ocurrir justo después de la última revisión manual, pasarían semanas completas hasta la siguiente revisión programada, durante las cuales miles de usuarios reales seguirían recibiendo respuestas de calidad ya degradada sin que nadie lo supiera; un sistema de monitoreo en tiempo real reduciría drásticamente esa ventana de exposición al problema, alertando mucho antes de que el daño se acumule tanto.

Lección 14 — Métricas de seguridad — tasa de bloqueo de guardrails, intentos de jailbreak

Puede revelar patrones de intentos de manipulación o uso indebido del sistema (como intentos de jailbreak), además de ayudar a identificar si los guardrails están siendo demasiado estrictos y bloqueando peticiones legítimas por error.

Un patrón estable que cambia repentinamente sugiere que algo específico está ocurriendo ahora que no ocurría antes, ya sea una campaña coordinada de intentos de manipulación recién iniciada por actores maliciosos, o un cambio no intencional en el comportamiento de los propios guardrails; en cualquier caso, ese cambio real merece investigarse en vez de descartarse como ruido aleatorio normal.

Lección 15 — Dashboards efectivos para sistemas de IA

Priorizar mostrar las métricas más accionables y relevantes de forma clara (calidad, costo, latencia, errores), evitando saturar el dashboard con demasiados detalles técnicos que dificulten identificar rápidamente si algo requiere atención.

Con decenas de métricas técnicas de bajo nivel compitiendo visualmente por atención, la señal realmente importante (como una caída en la calidad de las respuestas) podría perderse entre todo ese ruido visual adicional; un dashboard más enfocado en las métricas clave realmente accionables facilita que el equipo identifique de un vistazo rápido si algo requiere atención inmediata, sin esa sobrecarga de información secundaria distrayendo.

Módulo 4: Costos y rendimiento

Lección 16 — Observabilidad de costos — tokens, llamadas, por usuario/feature

Permite identificar exactamente qué usuarios o qué funcionalidades específicas están generando la mayor parte del costo, facilitando decisiones informadas sobre optimización, precios o priorización de esfuerzo de ingeniería.

La cifra total agregada de \$10,000 no dice nada sobre dónde enfocar el esfuerzo de optimización; saber que una feature específica representa el 70% de ese costo total convierte un número abstracto en información directamente accionable, dirigiendo la atención del equipo exactamente hacia donde una optimización tendría el mayor impacto real posible.

Lección 17 — Atribución de costos en sistemas multi-modelo

Requiere rastrear qué porción específica del costo total de esa petición corresponde a cada modelo individual invocado, dado que cada uno puede tener un precio por token y un patrón de uso completamente distinto entre sí.

Aunque el modelo grande de generación cueste más por token individual, si el modelo pequeño de clasificación se invoca muchas más veces dentro del mismo flujo, su costo acumulado total podría ser sorprendentemente comparable o incluso mayor; sin un desglose real y específico por cada modelo, el equipo solo tendría suposiciones intuitivas (potencialmente equivocadas) sobre dónde realmente se concentra el costo.

Lección 18 — Latencia — percentiles y su impacto en experiencia de usuario

El promedio puede ocultar que un pequeño porcentaje de usuarios experimenta latencias mucho peores que la mayoría, mientras el p99 revela específicamente qué tan mala es la experiencia para ese segmento de peticiones más lentas.

Aunque el promedio de 500ms suene perfectamente razonable, ese número agregado no refleja que un 1% real de usuarios está esperando hasta 8 segundos completos por su respuesta, una experiencia mucho más frustrante de la que el promedio general sugeriría; el p99 es precisamente la métrica que revela esa cola de experiencias significativamente peores que quedaría oculta si solo se mirara el promedio general del sistema.

Lección 19 — Observabilidad de uso de GPU y recursos de inferencia

Permite identificar si la infraestructura de GPU está siendo usada eficientemente (ni sobrecargada causando latencia alta, ni infrautilizada desperdiciando capacidad costosa), informando decisiones de escalado o optimización de recursos.

Si la latencia alta coincide temporalmente con una utilización de GPU cercana al máximo, eso sugiere que la infraestructura de inferencia está saturada y es la causa probable; si en cambio la GPU muestra utilización normal mientras la latencia sigue siendo alta, eso apunta hacia otra causa distinta (como un cuello de botella en otra parte del sistema), una distinción que solo es posible al correlacionar ambas métricas juntas.

Lección 20 — Presupuestos y alertas de costos

Permite detectar un gasto inesperadamente alto (por un bug, un aumento repentino de tráfico, o un loop descontrolado) mientras todavía es temprano para investigar y corregir la causa, antes de que se acumule un costo mucho mayor durante todo el período de facturación.

Si el problema se detecta a mitad de mes gracias a la alerta, el equipo todavía tiene la mitad restante del período para investigar la causa y corregirla, limitando el gasto excesivo adicional acumulado; si en cambio el problema solo se descubre al recibir la factura completa al final del mes, todo ese gasto excesivo ya ocurrió por completo sin ninguna oportunidad real de haberlo limitado a tiempo.

Módulo 5: Alertas, anomalías y respuesta a incidentes

Lección 21 — Definir alertas significativas para sistemas de IA

Debe indicar una condición que realmente requiere acción humana (como una degradación de calidad sostenida o un gasto anormal), evitando dispararse por fluctuaciones normales y esperadas que no representan un problema real.

Cuando el equipo recibe alertas constantemente que resultan ser falsos positivos sin ningún problema real detrás, con el tiempo desarrolla el hábito de descartarlas automáticamente sin investigar; ese patrón de comportamiento aprendido hace que, cuando finalmente aparece una alerta que sí representa un problema genuino, se trate con la misma indiferencia que todas las anteriores, perdiendo su valor real como señal confiable de atención.

Lección 22 — Detección de anomalías en el comportamiento del modelo

Puede adaptarse a patrones normales que varían según el momento (como más tráfico en horario laboral) y detectar desviaciones genuinamente inusuales respecto a ese patrón esperado, en vez de un único umbral fijo que podría no ser apropiado en todos los contextos.

Un umbral fijo de 1000 peticiones por minuto no sabe si ese número es normal o anormal según el contexto; durante el horario pico legítimo del negocio, superar esa cifra podría ser completamente esperado y no representar ningún problema real, mientras un sistema que aprendió el patrón habitual de tráfico según la hora del día reconocería ese aumento como parte del comportamiento normal esperado, evitando una alerta innecesaria.

Lección 23 — Runbooks para incidentes específicos de IA

Pasos específicos como verificar si hubo un cambio reciente de modelo o prompt, revisar métricas de drift, consultar ejemplos recientes de respuestas problemáticas capturadas por el sistema de evaluación, y considerar un rollback a una versión anterior conocida como estable.

Durante la presión de un incidente real, tener ya definidos los pasos específicos de investigación (qué revisar primero, qué preguntas hacer) ahorra el tiempo valioso que se perdería si cada persona tuviera que decidir desde cero por dónde empezar; el runbook captura el conocimiento acumulado de investigaciones anteriores similares, acelerando y haciendo más consistente la respuesta del equipo.

Lección 24 — On-call para sistemas de IA — qué es diferente

Comprensión de conceptos específicos de IA (como qué es un prompt, qué es drift, cómo interpretar métricas de calidad) para poder diagnosticar y responder efectivamente ante un incidente relacionado con la calidad del modelo, no solo con la infraestructura técnica.

Reiniciar un servidor caído es una habilidad tradicional de infraestructura bien conocida, pero una alerta de 'drift detectado' requiere entender qué es el drift, por qué ocurre y qué acciones son apropiadas en respuesta (como investigar los datos recientes o considerar un reentrenamiento), un conocimiento específico de IA que las habilidades tradicionales de operar servidores no cubren por sí solas.

Lección 25 — Post-mortems de incidentes de IA

Qué señales de observabilidad de calidad (si existían) podrían haber permitido detectar el problema antes, y qué cambios al sistema de evaluación o monitoreo de calidad se necesitan para detectar un problema similar más rápido en el futuro.

Esa pregunta específica dirige el análisis hacia identificar exactamente qué brecha existía en el sistema de monitoreo actual (una métrica que no se estaba capturando, un umbral mal calibrado); esa identificación concreta es lo que permite convertir el aprendizaje del post-mortem en una mejora real y específica del sistema de observabilidad, en vez de quedarse solo en entender qué pasó sin actuar sobre esa brecha detectada.

Módulo 6: Producción avanzada

Lección 26 — Feedback humano como señal de observabilidad

Provee una señal directa y real de percepción de calidad desde la perspectiva del usuario final, complementando las métricas de evaluación automatizada con una fuente de verdad adicional basada en experiencia real.

Si la evaluación automatizada califica sistemáticamente como 'buena' una respuesta que los usuarios reales marcan repetidamente como 'no útil', esa discrepancia sugiere que el criterio automatizado podría estar midiendo un aspecto de calidad distinto de lo que realmente importa para los usuarios reales en la práctica, una brecha valiosa de investigar para mejorar el propio sistema de evaluación.

Lección 27 — Observabilidad para cumplimiento y auditoría (compliance)

Mantener un registro auditable y trazable de las decisiones del sistema (qué información se usó, qué respuesta se generó, cuándo), de forma que pueda reconstruirse y justificarse ante un auditor externo o regulador cuando sea necesario.

Una decisión que niega crédito a una persona tiene un impacto real y directo sobre su vida, y la regulación del sector financiero típicamente exige poder justificar y explicar cómo se llegó a esa decisión específica si esa persona (o un regulador) lo cuestiona después; sin un registro auditable completo, sería imposible reconstruir y justificar esa decisión meses después de que realmente ocurrió.

Lección 28 — Observabilidad multi-agente — visibilidad de sistemas complejos

Requiere capturar no solo las decisiones individuales de cada agente, sino también la comunicación y coordinación entre ellos, para entender cómo la interacción conjunta llevó al resultado final del sistema completo.

Cada agente individual podría haber tomado una decisión perfectamente razonable dentro de su propio contexto aislado, pero el problema real podría estar en cómo un agente interpretó o usó incorrectamente la información que otro agente le comunicó; sin capturar esa comunicación específica entre ellos, la causa raíz real del resultado final incorrecto del sistema completo permanecería oculta, aunque cada decisión individual pareciera correcta vista por separado.

Lección 29 — Cultura de observabilidad — de reactivo a proactivo

Un equipo reactivo solo revisa la observabilidad cuando ya ocurrió un problema reportado por usuarios, mientras uno proactivo revisa regularmente tendencias y métricas para detectar señales tempranas de degradación antes de que se convierta en un incidente reportado.

Para que una queja de usuario llegue al equipo, el problema ya tuvo que haber afectado negativamente a esa persona real primero; un equipo que revisa proactivamente las tendencias de sus métricas de calidad podría detectar la degradación empezando a ocurrir antes de que se acumulen suficientes quejas, limitando cuántos usuarios reales terminan siendo afectados por ese mismo problema.

Lección 30 — Proyecto final: stack de observabilidad de IA completo listo para producción

Tracing de extremo a extremo con OpenTelemetry + monitoreo continuo de calidad con evaluación automatizada + observabilidad de costos y latencia con percentiles + alertas significativas calibradas +

runbooks específicos + feedback humano integrado.

Intenta mencionar al menos dos pilares: tracing de extremo a extremo, monitoreo continuo de calidad, observabilidad de costos/latencia o alertas significativas con runbooks.